Heart Disease Risk Prediction

End-to-End MLOps Pipeline: Development, CI/CD, Deployment and Monitoring

Name	Abhishek Pagariya
BITS ID	2025ae05133
Subject	Machine Learning Operations (MLOps)
Subject Code	AIMLCZG523
Assignment	Assignment 1 (50 marks)
Institute	BITS Pilani - Work Integrated Learning Programmes
Submission	October 2026

Repository: github.com/abhishekssig/heart-disease-mlops

Environment: BITS Prayogshala virtual lab (MLOPS), Rocky Linux, Python 3.12, Docker 28, Minikube 1.35

Contents

Contents 2
1. Problem Statement and Objectives 3
2. Dataset 3
3. Data Cleaning and Exploratory Data Analysis 3
3.1 Cleaning 3
3.2 Exploratory analysis 4
4. Feature Engineering 5
5. Model Development and Selection 6
6. Experiment Tracking with MLflow 7
7. Model Packaging and Reproducibility 8
8. Testing and CI/CD 8
8.1 Unit tests 8
8.2 GitHub Actions pipeline 9
9. API and Containerisation 10
10. Deployment on Kubernetes 11
11. Monitoring and Logging 11
11.1 Logging 11
11.2 Metrics 12
12. Architecture 13
13. Setup and Run Instructions 13
14. Challenges and Lessons Learned 13
15. Limitations and Future Work 14
16. Deliverables 14

1. Problem Statement and Objectives

The goal of this assignment is to build a classifier that predicts whether a patient has heart disease from routine clinical measurements, and to take it all the way to a monitored API running on Kubernetes. The machine learning part is deliberately small; most of the effort went into making the pipeline reproducible, tested, automated and observable.

Concretely, I set the following objectives:

- A single command reproduces data download, cleaning, EDA, training and evaluation from a clean clone.
- Every training run is tracked in MLflow with parameters, metrics, plots and the model itself.
- The model is shipped as one scikit-learn pipeline so preprocessing at serving time is identical to training.
- GitHub Actions lints, tests, trains, builds the Docker image and calls the running container on every push.
- The API runs on Minikube behind a LoadBalancer and Ingress, scales with an HPA and is monitored with Prometheus and Grafana.

All of the steps in this report were executed on the BITS Prayogshala MLOPS virtual lab, and the screenshots come from that VM and from the GitHub Actions run on the repository.

2. Dataset

I used the **UCI Heart Disease dataset, Cleveland subset** (processed.cleveland.data): 303 patients and 13 attributes. The original target `num` ranges from 0 (no disease) to 4; I collapsed it to a binary label where 1-4 means disease is present.

Group	Features	Treatment
Numeric	age, trestbps (resting BP), chol, thalach (max HR), oldpeak (ST depression)	median impute + standard scale
Binary	sex, fbs (fasting sugar > 120), exang (exercise angina)	most-frequent impute
Categorical	cp (chest pain), restecg, slope, ca (vessels), thal	most-frequent impute + one-hot

The raw file is downloaded by `python -m heartml.data`, which writes `data/raw/processed.cleveland.data` and the cleaned `data/processed/heart_clean.csv`. The download URL is kept in one config module so the script works the same locally, on the VM and in CI.

3. Data Cleaning and Exploratory Data Analysis

3.1 Cleaning

- Missing values are encoded as `?` in the raw file and parsed as NaN. Only **ca (4 rows)** and **thal (2 rows)** are affected.
- The target is binarised and duplicates are dropped (none were found).
- Numeric values outside plausible clinical ranges (e.g. cholesterol below 100) are set to missing instead of dropping the patient. No row in this dataset triggered it, but the check protects the API from bad input.

- Imputation is not done in the cleaning step on purpose. It lives inside the model pipeline so the same medians learned on training data are applied to API requests.

3.2 Exploratory analysis

Running `python -m heartml.eda` produces six figures and a JSON summary. The classes are close to balanced: 164 patients without disease and 139 with (46% positive), so accuracy is a reasonable metric, though I still report precision, recall and ROC-AUC.

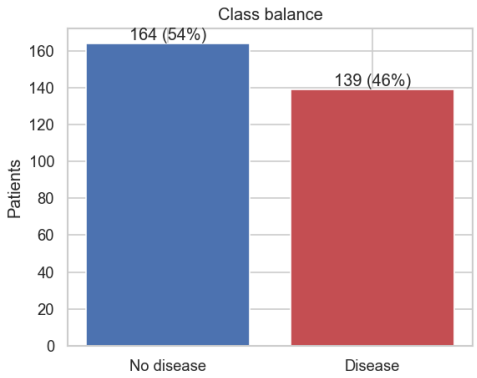


Figure 1: Class balance of the cleaned dataset

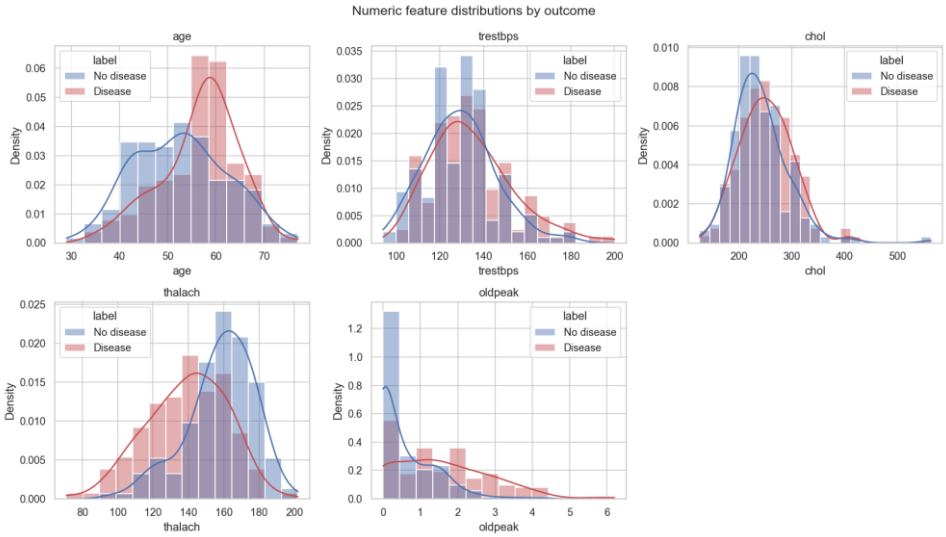


Figure 2: Distribution of numeric features split by outcome

The histograms show clear separation for maximum heart rate (thalach is lower in diseased patients) and ST depression (oldpeak is higher). Cholesterol and resting blood pressure overlap heavily between classes.

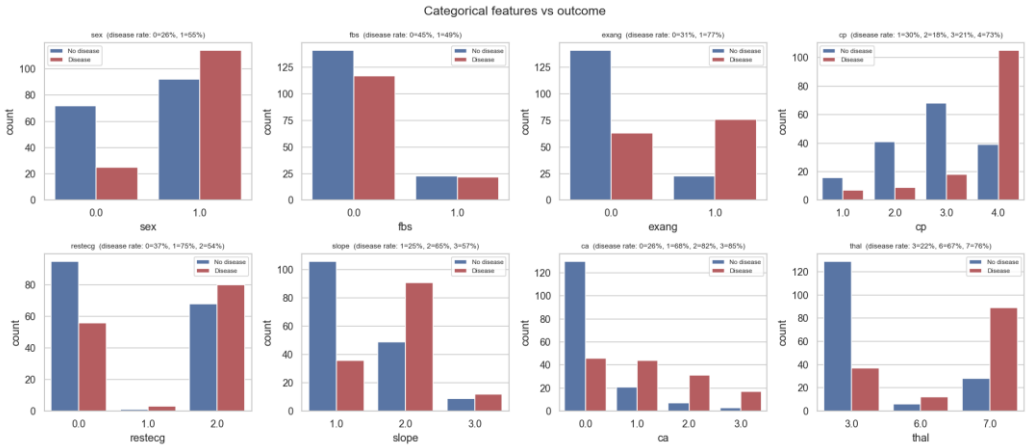


Figure 3: Categorical features versus outcome, with disease rate per category

Among categorical features, asymptomatic chest pain (cp = 4) has a 73% disease rate, exercise-induced angina 77%, and any coloured major vessel (ca >= 1) 68-85%. Fasting blood sugar shows almost no difference (45% vs 49%).

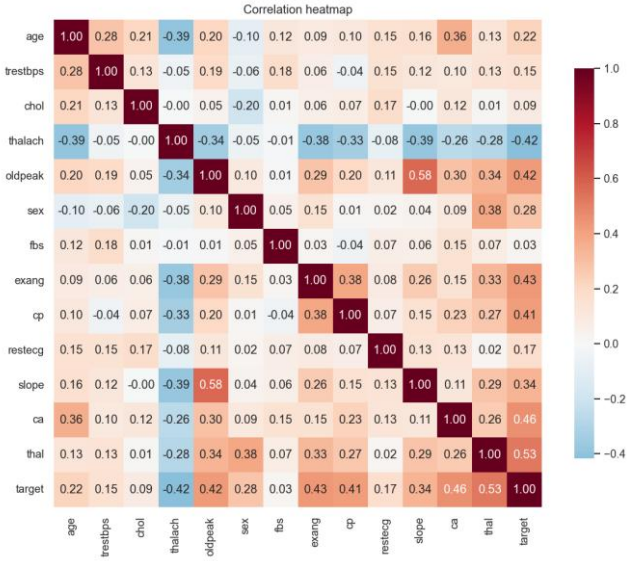


Figure 4: Correlation heatmap including the target

The strongest correlations with the target are thal (0.53), ca (0.46), exang (0.43), oldpeak (0.43), cp (0.41) and thalach (-0.42). Because ca and thal are coded categories rather than measurements, I one-hot encode them instead of scaling them.

4. Feature Engineering

All preprocessing is a single scikit-learn `Pipeline` defined in `heartml/features.py`. It has two stages:

Clinical features. A custom transformer adds three domain-driven columns:

- **hr_reserve_pct** = thalach / (220 - age): the share of the age-predicted maximum heart rate the patient actually reached during the stress test.
- **st_per_hr** = oldpeak / thalach x 100: ST depression normalised by how hard the heart worked.
- **bp_stage2** = resting BP >= 140 mmHg, the clinical stage-2 hypertension threshold.

Column transformer. Numeric features (including the two ratios) are median-imputed and standardised, binary features are mode-imputed, and categorical features are mode-imputed and one-hot encoded with unknown categories ignored. The output has 28 columns.

Packaging preprocessing and the estimator together means the saved artefact accepts raw patient JSON, which removes a whole class of training/serving skew bugs.

5. Model Development and Selection

I trained three model families with `GridSearchCV` on an 80/20 stratified split (242 train, 61 test, seed 42). Each search used 5-fold stratified cross-validation, scored accuracy, precision, recall, F1 and ROC-AUC, and refit on ROC-AUC.

Model	Search space	Candidates	Best parameters
Logistic regression	C {0.01, 0.1, 0.3, 1, 3}, penalty {l1, l2}, class_weight {None, balanced}	20	C=0.3, l1, balanced
Random forest	n_estimators {200, 400}, max_depth {None, 4, 8}, min_samples_leaf {1, 3, 5}, max_features {sqrt, 0.5}	36	400 trees, depth None, leaf 3, sqrt
Gradient boosting	n_estimators {100, 200}, learning_rate {0.03, 0.1}, max_depth {2, 3}, subsample {0.8, 1.0}	16	100 trees, lr 0.03, depth 2, subsample 0.8

Model	CV ROC-AUC	CV accuracy	Test ROC-AUC	Test accuracy	Test precision	Test recall	Test F1
Logistic regression	0.906 +/- 0.013	0.839	0.968	0.885	0.839	0.929	0.881
Random forest	0.892 +/- 0.025	0.801	0.960	0.902	0.867	0.929	0.897
Gradient boosting	0.883 +/- 0.024	0.814	0.959	0.902	0.867	0.929	0.897

Selection. I selected the model by mean cross-validated ROC-AUC on the training split only, so the test set never influences the choice. Logistic regression won (0.906) and also had the smallest variance across folds (0.013). The tree ensembles score slightly higher test accuracy, but on 61 test patients a difference of one prediction is 1.6 percentage points, so I did not treat that as meaningful. A simpler, more stable and interpretable model is the better choice for 303 rows.

On the test set the selected model gets 54 of 61 right: 26 of 28 diseased patients are caught (recall 0.93), with 5 false positives and 2 false negatives. For a screening use-case, missing a sick patient is worse than an extra referral, so this trade-off is acceptable.

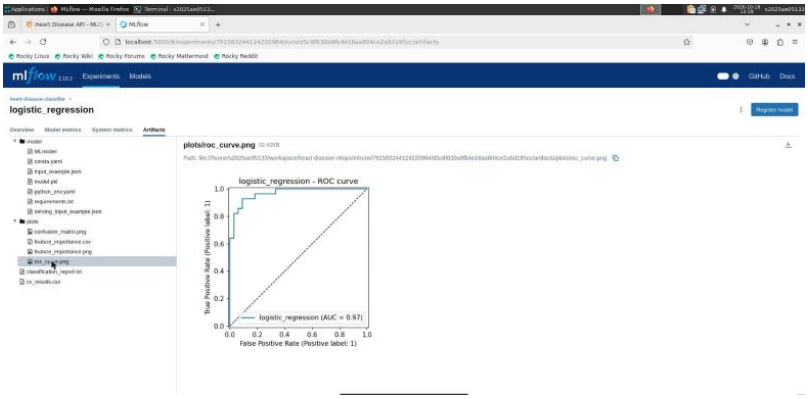


Figure 5: ROC curve of the selected model (AUC 0.97), logged as an MLflow artefact

The L1 penalty zeroed 14 of 28 coefficients. The largest effects are ca = 0 (no coloured vessels, strongly protective), asymptomatic chest pain (cp = 4), male sex, reversible defect thal = 7 and normal thal = 3 (protective). The engineered st_per_hr survived with a positive weight, while hr_reserve_pct was dropped because it is redundant with thalach and age.

6. Experiment Tracking with MLflow

Training logs to the experiment `heart-disease-classifier`. Each run of `python -m heartml.train` creates:

- One parent run per model family with dataset sizes, grid size, best parameters, mean and standard deviation of every CV metric, all test metrics, a confusion matrix, ROC curve, feature-importance plot and CSV, a classification report, the full cv_results.csv, and the fitted pipeline as an MLflow sklearn model with signature and input example.
- One nested child run per grid candidate (72 in total) with its parameters and CV scores, so every configuration tried is visible.
- A model-selection run with the comparison table and the chosen model.

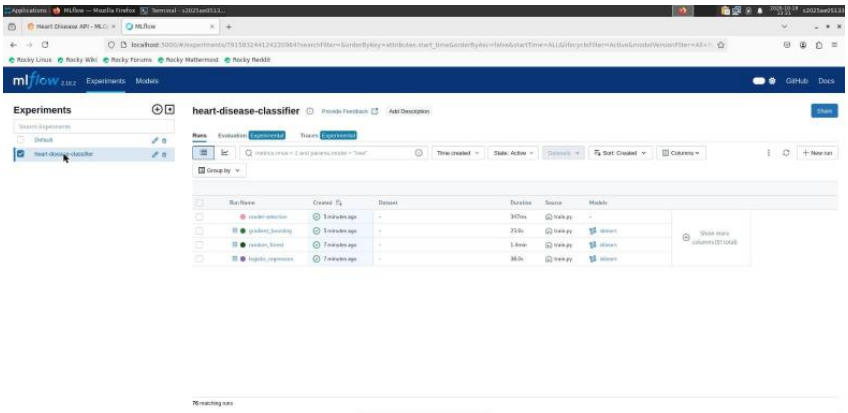


Figure 6: MLflow experiment on Prayogshala: 76 runs (3 parents, 72 candidates, 1 selection)

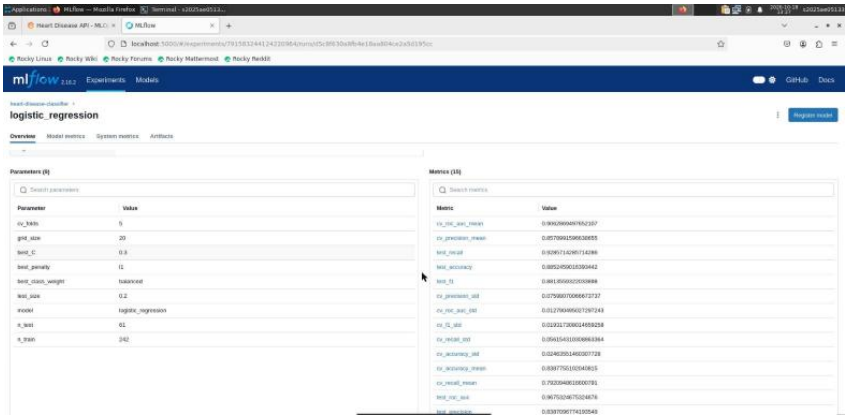


Figure 7: Parameters and metrics logged for the logistic regression run

7. Model Packaging and Reproducibility

- `models/model.joblib` - the full preprocessing + model pipeline.
- `models/metadata.json` - model name, MLflow run id, best parameters, metrics, Python and scikit-learn versions, decision threshold and training timestamp.
- `requirements.txt` pins every dependency; `requirements-serve.txt` contains only the runtime subset for the Docker image, with matching versions so the pickle loads cleanly.
- Fixed random seeds for the split, the CV folds and the models. The run on Prayogshala reproduced exactly the same metrics as my laptop and the CI runner.
- `scripts/setup_vm.sh` creates the virtualenv and checks tools; `scripts/run_pipeline.sh` runs lint, tests, data, EDA, training and a batch prediction.

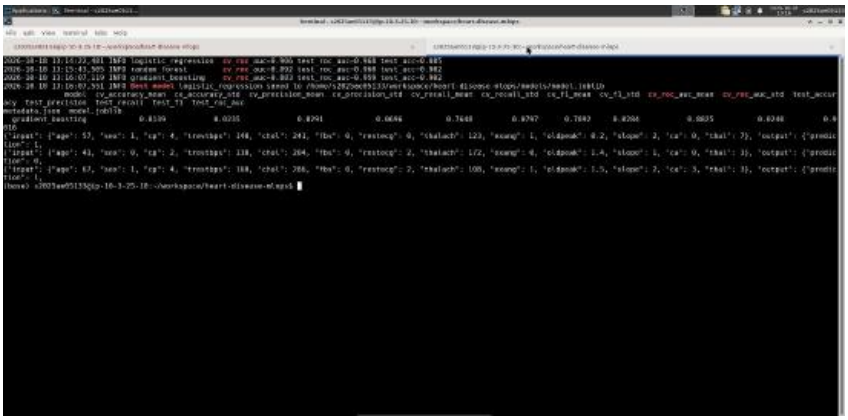


Figure 8: Full pipeline run on the Prayogshala VM: three models trained, best model saved, sample predictions

8. Testing and CI/CD

8.1 Unit tests

The `tests/` folder has 23 pytest tests that run on a synthetic dataset, so they need no network access:

File	What it checks
test_data.py	binary target, column order, duplicate removal, out-of-range values become NaN, '?' parsed as missing
test_features.py	engineered feature values, input not mutated, no NaN after preprocessing, unseen categories, scaling

File	What it checks
test_model.py	LR and RF train and output valid probabilities, model learns signal, metric helper, save/load round trip, missing optional fields
test_api.py	/health, /predict, validation errors (422), batch endpoint, Prometheus metrics, 503 when the model file is missing

8.2 GitHub Actions pipeline

The workflow `.github/workflows/ci.yml` runs on every push and pull request. Jobs are chained, so any failure stops everything after it:

Job	Steps	Artefacts
lint	flake8 on heartml and tests	flake8 report
test	pytest with coverage	JUnit XML, coverage XML
train	download, EDA, full training, quality gate (test ROC-AUC >= 0.80), batch inference smoke test	model, mlruns, figures, training log, cleaned data
docker	build image with trained model, run container, call /health, /predict and /metrics, assert response	container log, prediction JSON, image tarball

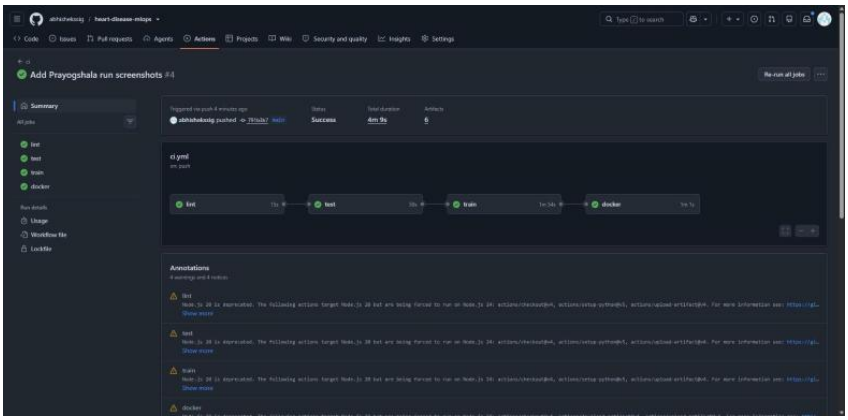


Figure 9: Successful GitHub Actions run: lint, test, train and docker

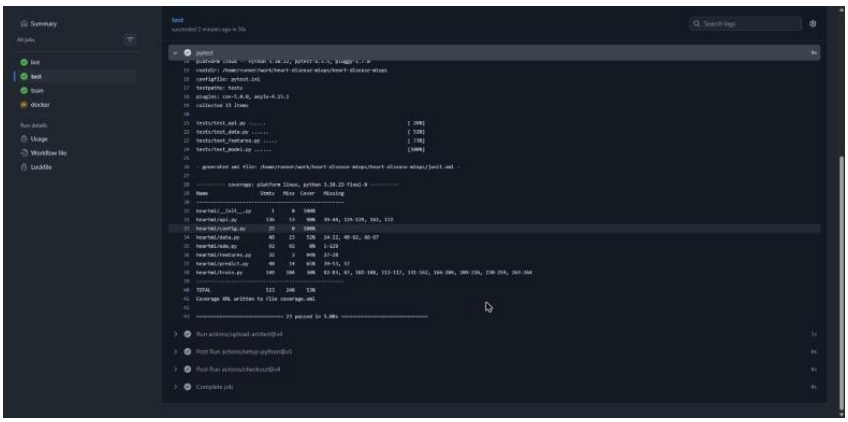


Figure 10: 23 tests passing in CI with coverage report

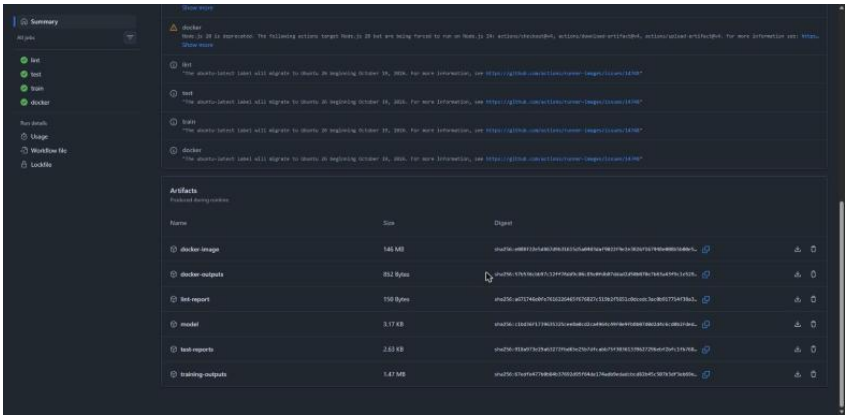


Figure 11: Artefacts uploaded by the workflow run

9. API and Containerisation

The API is built with FastAPI (`heartml/api.py`). Requests are validated with Pydantic against clinical ranges, so an age of 250 or a missing field returns HTTP 422 before it reaches the model.

Method	Path	Purpose
GET	/health	readiness / liveness; 503 if the model is not loaded
GET	/model-info	model name, parameters and metrics from metadata.json
POST	/predict	one patient -> prediction, label, probability_disease, confidence
POST	/predict/batch	up to 500 patients in one call
GET	/metrics	Prometheus metrics

```
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d
@sample_input.json
{"prediction":1,"label":"Heart
disease","probability_disease":0.8128,"confidence":0.8128}
```

The Dockerfile uses `python:3.10-slim`, installs only the serving requirements, runs as a non-root user and defines a HEALTHCHECK on /health. On the VM the image built to 480 MB and the container reported healthy within six seconds.

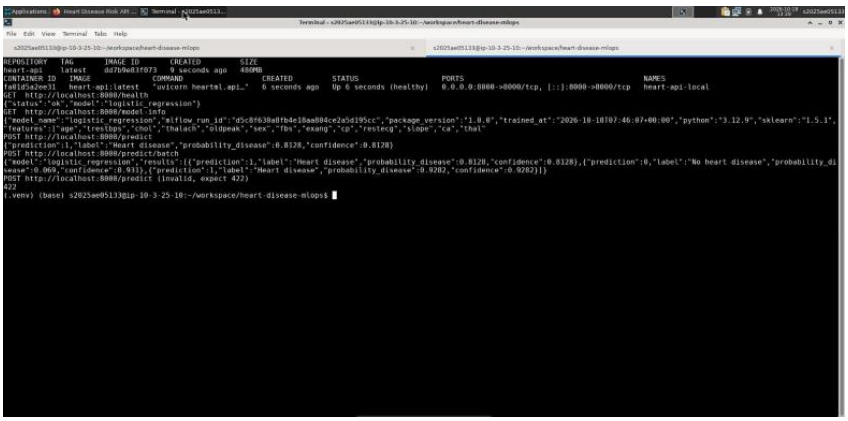


Figure 12: Docker image built and run on Prayogshala; smoke test of all endpoints

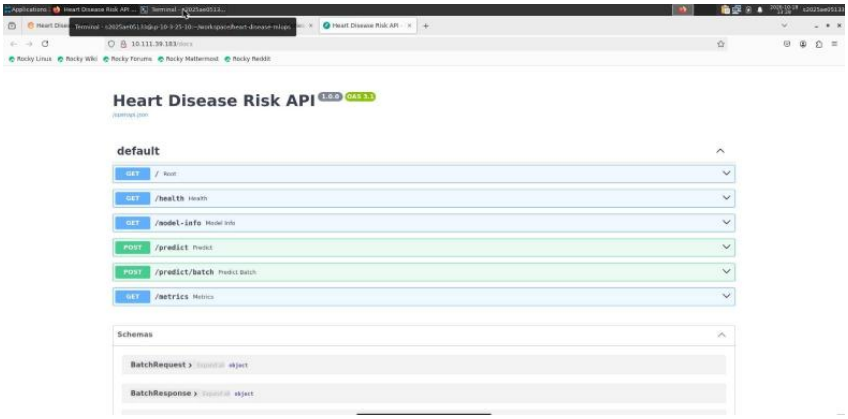


Figure 13: Auto-generated OpenAPI documentation served by the deployed API

10. Deployment on Kubernetes

I deployed to Minikube (Kubernetes 1.32, Docker driver) on the Prayogshala VM with `scripts/deploy_minikube.sh`. The image is built straight into Minikube's Docker daemon, so no registry is needed.

Manifest	Resource	Key settings
namespace.yaml	Namespace heart	isolates the app and monitoring
deployment.yaml	Deployment heart-api	2 replicas, rolling update with maxUnavailable 0, readiness and liveness probes on /health, CPU/memory requests and limits, Prometheus scrape annotations
service.yaml	Service (LoadBalancer)	port 80 -> 8000; external IP via minikube tunnel
ingress.yaml	Ingress (nginx)	host heart-api.local
hpa.yaml	HorizontalPodAutoscaler	2-5 replicas at 70% CPU

I verified the API three ways: through the NodePort, through the LoadBalancer external IP (10.111.39.183) and through the Ingress host. During a load test of 1,000 requests the HPA measured 77% CPU and scaled the deployment from 2 to 3 pods on its own.

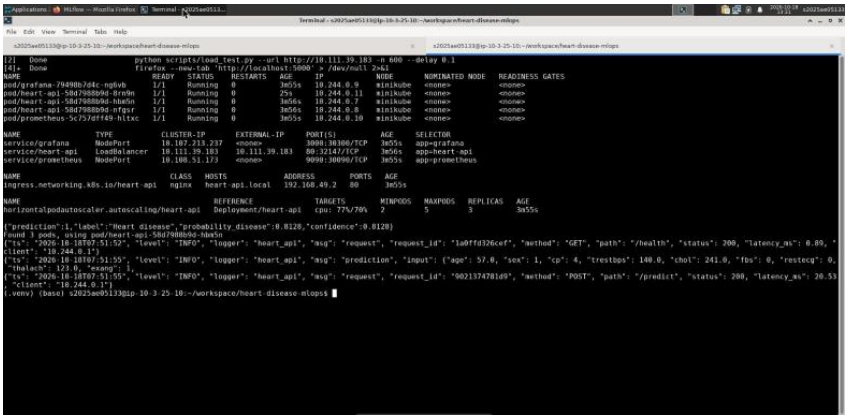


Figure 14: Pods, LoadBalancer, Ingress and HPA scaled to 3 replicas; structured JSON request logs

11. Monitoring and Logging

11.1 Logging

Every request produces one JSON log line with a request id, method, path, status, latency and client address, and every prediction logs its input and output. The request id is also returned in the x-request-id header so a client error can be matched to a log line. Logs go to stdout, which is what kubectl logs and any log collector read.

11.2 Metrics

Metric	Type	Use
heart_api_requests_total	counter (method, endpoint, status)	traffic and error rate
heart_api_request_latency_seconds	histogram (endpoint)	p50 / p95 / p99 latency
heart_api_predictions_total	counter (label)	class balance of predictions
heart_api_prediction_confidence	histogram	model confidence over time
heart_api_disease_probability	histogram	simple drift signal
heart_api_model_loaded	gauge	model availability

Prometheus discovers the API pods through Kubernetes service discovery and the scrape annotations, so new replicas are picked up automatically. Alert rules cover a target being down, p95 latency above 500 ms and any 5xx responses. Grafana is provisioned with the datasource and dashboard from the repository, so it shows data with no manual setup.

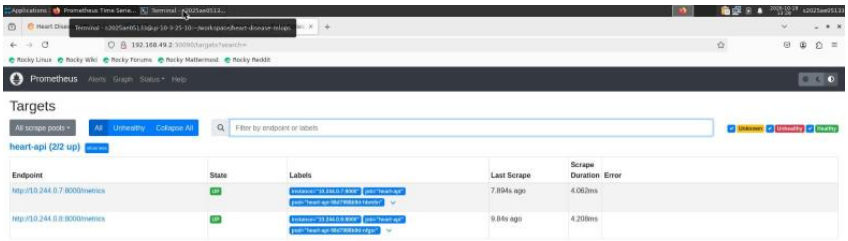


Figure 15: Prometheus scraping both API pods

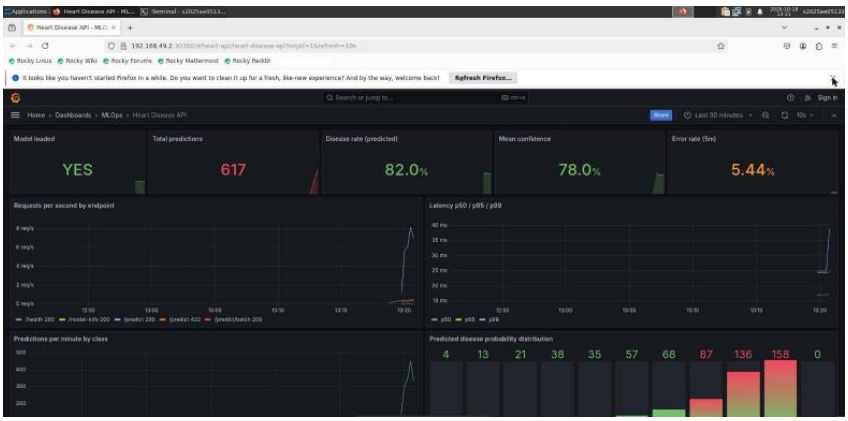


Figure 16: Grafana dashboard during the load test

In the dashboard the error rate of about 5% is expected: the load generator deliberately sends 5% invalid requests to show that validation errors are counted and visible.

12. Architecture

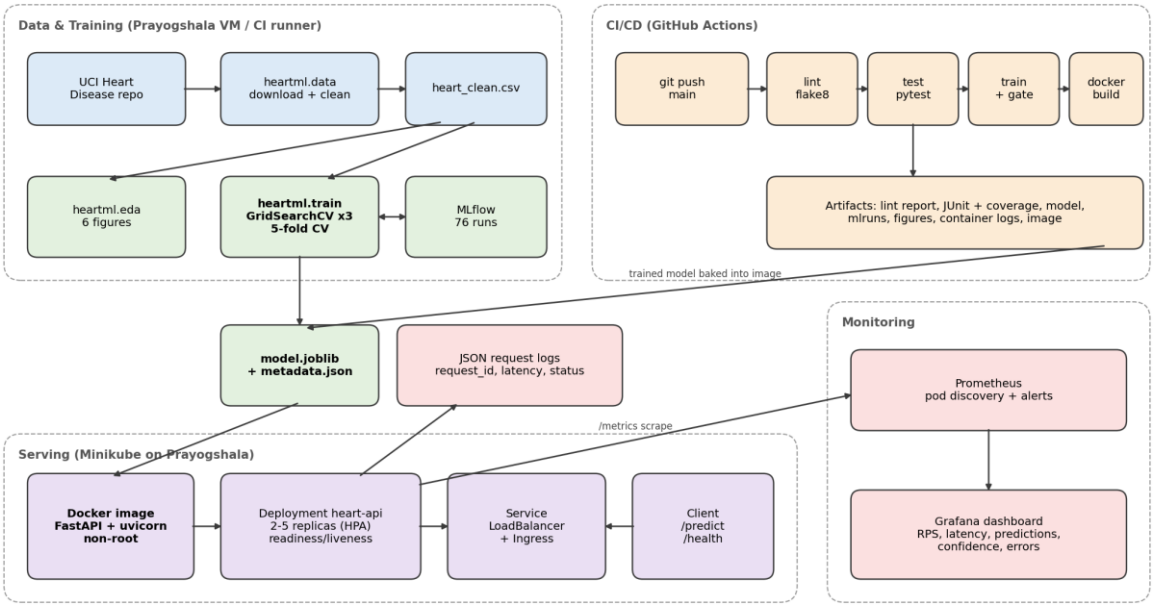


Figure 17: End-to-end architecture of the solution

Code is pushed to GitHub, which triggers the CI pipeline. The same package code runs data preparation and training locally, on Prayogshala and in CI, logging to MLflow and producing a versioned pipeline file. That file is baked into the Docker image, which runs as a Deployment on Minikube behind a LoadBalancer and Ingress. Prometheus scrapes every pod and Grafana visualises the metrics.

13. Setup and Run Instructions

```
cd ~/workspace
git clone https://github.com/abhishekssig/heart-disease-mlops.git
cd heart-disease-mlops
bash scripts/setup_vm.sh
bash scripts/run_pipeline.sh
mlflow ui --backend-store-uri ./mlruns --port 5000
docker build -t heart-api:latest . && docker run -p 8000:8000 heart-api:latest
bash scripts/deploy_minikube.sh
minikube tunnel &
kubectl -n heart get svc heart-api
python scripts/load_test.py --url http://<EXTERNAL-IP> -n 300
```

Grafana is on NodePort 30300 and Prometheus on 30090 of the Minikube IP. Because the API runs on a lab VM it is not publicly reachable; the commands above are the access instructions.

14. Challenges and Lessons Learned

- **Lab constraints.** Prayogshala wipes everything outside ~/workspace when the VM stops and ends idle sessions after 15 minutes. Keeping everything in Git and making setup a single script meant a fresh VM was back to a running cluster in about ten minutes.

- **Dependency pinning.** matplotlib 3.9.1 had no Windows wheel, so pip tried to compile it; pinning 3.9.2 fixed it. Keeping the serving requirements a strict subset of the training ones avoided pickle version mismatches.
- **Metrics with multiple workers.** Running uvicorn with several workers would give each process its own Prometheus counters. I kept one worker per container and scale with replicas instead.
- **Test-set discipline.** It is tempting to pick the model with the best test accuracy (the ensembles). Selecting on CV ROC-AUC keeps the test set honest.

15. Limitations and Future Work

- 303 patients from a single hospital; validating on the Hungarian, Swiss and VA subsets would test generalisation.
- The decision threshold is fixed at 0.5; a clinical deployment would tune it for higher recall.
- The model is baked into the image. Using the MLflow Model Registry and loading the model at start-up would allow model releases without rebuilding.
- Drift monitoring is limited to the predicted-probability histogram; a scheduled job comparing feature distributions to the training data would be the next step.

16. Deliverables

Item	Location
Code repository	github.com/abhishekssig/heart-disease-mlops
Cleaned data + download script	data/processed/heart_clean.csv, heartml/data.py
EDA / training / inference	heartml/eda.py, heartml/train.py, heartml/predict.py, notebooks/
Tests	tests/
CI workflow	.github/workflows/ci.yml
Dockerfile and compose	Dockerfile, docker-compose.yml
Kubernetes manifests	k8s/, k8s/monitoring/
Monitoring config	monitoring/
Screenshots	screenshots/
Video	pipeline_demo (submitted with this report)